

Exercise: Deep Copying a Dynamic Array

Background & Core Concepts: Shallow Copy vs. Deep Copy

When working with dynamic memory on the heap, simply assigning one pointer to another ($p2 = p1$) performs a shallow copy. Both pointers hold the exact same memory address. Modifying data through $p2$ alters $p1$, and calling `delete[] p1` leaves $p2$ as a dangerous dangling pointer.

To create an independent duplicate, you must perform a deep copy:

- Request a brand-new memory block on the heap using `new`.
- Copy each element value individually from the source block to the new destination block.

Shallow Copy:

```
p1 ----> [ 10.5 | 20.2 | 30.8 ]
           ^
p2 -----|
```

Deep Copy:

```
src  ----> [ 10.5 | 20.2 | 30.8 ]
dest ----> [ 10.5 | 20.2 | 30.8 ] (Newly allocated memory block)
```

Pointer Arithmetic as Syntax Sugar

In C++, bracket notation (`arr[i]`) is just "syntax sugar" for pointer arithmetic:

$$\text{arr}[i] \Leftrightarrow *(\text{arr} + i)$$

Adding integer i to pointer `arr` shifts the memory address forward by $i \times \text{sizeof}(\text{type})$ bytes. Dereferencing with `*` accesses the value stored at that shifted address.

2. Lab Problem Statement & Constraints

Write two implementations of `deepCopyVector` that allocate a new dynamic array of floats on the heap and perform an element-wise deep copy from `src`. Subsequently, derive and compare the worst-case Time Complexity $O(f(n))$ and Auxiliary Space Complexity $O(g(n))$ for both solutions.

Constraints

- **No Bracket Notation:** You are **NOT** allowed to use square brackets ([]) inside the function body, except for the initial heap allocation (new float[size]).
- **Null Pointer Safety:** Return nullptr if src is nullptr or if size <= 0.
- **Memory Management:** Ensure the calling function receives the base address of the newly allocated array so it can be safely freed with delete[].

Step-by-Step Implementation Guide

Solution 1: Base Address + Index Offset

This approach keeps the original base pointers stationary and calculates element addresses on each iteration using an offset integer i.

- Step 1: Validate input (src != nullptr and size > 0).
- Step 2: Allocate memory on the heap: float* dest = new float[size];.
- Step 3: Set up a for loop running from index i = 0 to size - 1.
- Step 4: Calculate source address (src + i) and destination address (dest + i).
- Step 5: Copy element value using dereferencing: *(dest + i) = *(src + i);.
- Step 6: Return base pointer dest.

Solution 2: Moving Worker Pointers

This approach creates temporary "worker" pointers that step through memory addresses increment-by-increment (ptr++).

- Step 1: Validate input (src != nullptr and size > 0).
- Step 2: Allocate memory on the heap: float* dest = new float[size];.
- Step 3: Initialize worker pointers pointing to the start of each block:
 - const float* srcPtr = src;
 - float* destPtr = dest;
- Step 4: Loop size times:
 - Copy current element: *destPtr = *srcPtr;
 - Advance both worker pointers to the next element: srcPtr++; destPtr++;
- Step 5: Return the original base pointer dest (Warning: returning destPtr would return a pointer to the end of the array, causing memory corruption when freed).

Exercise: Book Store Inventory System

Objective

Practice raw pointer manipulation, dynamic heap allocation with new/delete, accessing struct members via the arrow operator (->), and managing an array of pointers.

Problem Statement

You are building an inventory management system using raw pointers. Define a Book struct and write functions to dynamically allocate, search, modify, and free book objects stored in an array of pointers.

Core Concepts & Mental Model

In this lab, you manage an array of pointers pointing to Book objects allocated on the heap:

- **The Heap (new):** When you call new Book, C++ allocates memory on the heap and returns its address.
- **Array of Pointers (Book* inventory[]):** Instead of storing entire Book objects directly in an array, inventory stores memory addresses (pointers) pointing to those books.
- **Arrow Operator (->):** When accessing fields through a pointer, use ptr->field instead of obj.field.
- **Null Pointers (nullptr):** nullptr represents a pointer that does not point to any valid memory location. You use nullptr to:
 - **Initialize:** Ensure unassigned pointer array slots don't hold garbage addresses.
 - **Safety Checks:** Check if (ptr != nullptr) before accessing members using -> to prevent program crashes (segmentation faults).
 - **Cleanup:** Reset pointers to nullptr immediately after calling delete so they don't become dangling pointers.
- **Reference Parameters (int& count):** Modifying count inside functions like addBook or deleteBook requires passing count by reference (&) so the updated total persists across function calls.

Requirements

Struct Definition: Create a Book struct with members:

- title (std::string)
- price (double)
- stockQuantity (int)

Functions to Implement:

1. Function: **addBook**

Signature: void addBook(Book* inventory[], int& count, const std::string& title, double price, int stock)

- Step 1 (Allocate): Use the new keyword to dynamically create a new Book instance on the heap.
- Step 2 (Assign): Store the resulting heap memory address inside inventory[count].
- Step 3 (Increment Count): Increment count by 1 (count++) so the host program tracks the updated total number of active items.

2. Function: **printBook**

Signature: void printBook(const Book* b)

- Step 1 (Safety Check): Check if b == nullptr. If true, print an error message or return early to avoid dereferencing a null pointer.
- Step 2 (Dereference & Print): Access the struct members using arrow notation (b->title, b->price, b->stockQuantity).
- Step 3 (Format): Output the title, price, and stock quantity clearly to std::cout.

3. Function: **applyDiscount**

Signature: void applyDiscount(Book* b, double percent)

- Step 1 (Safety Check): Check if b == nullptr before attempting to access memory.
- Step 2 (Calculate Discount): Modify the object's price directly using arrow syntax: b->price -= b->price * (percent / 100.0).

4. Function: **modifyBook**

Signature: bool modifyBook(Book* inventory[], int count, const std::string& title, double newPrice, int newStock)

- Step 1 (Linear Search): Loop through the array from index i = 0 to count - 1.
- Step 2 (Match Check): For each slot, ensure inventory[i] != nullptr and compare inventory[i]->title == title.
- Step 3 (Update): If a matching title is found:
 - Update inventory[i]->price = newPrice

- Update inventory[i]->stockQuantity = newStock
 - Return true immediately to indicate success.
- Step 4 (Fail Return): If the loop completes without finding the title, return false.

5. Function: deleteBook

Signature: bool deleteBook(Book* inventory[], int& count, const std::string& title)

- Step 1 (Search): Loop from i = 0 to count - 1 to locate the target title.
- Step 2 (Deallocate Memory): Once found, release the allocated heap memory by calling delete inventory[i].
- Step 3 (Shift Pointers Left): Shift all remaining pointer elements one position to the left using an inner loop from j = i to count - 2: inventory[j] = inventory[j + 1].
- Step 4 (Clear Vacant Slot): Set the old last element slot inventory[count - 1] = nullptr to ensure no dangling pointer remains.
- Step 5 (Update Count & Return): Decrement count (count--) and return true.
- Step 6 (Fail Return): If the title does not exist in the array, return false.

6. Function: freeInventory

Signature: void freeInventory(Book* inventory[], int& count)

- Step 1 (Loop Through Array): Iterate through all occupied slots from i = 0 to count - 1.
- Step 2 (Deallocate Heap Objects): Call delete inventory[i] on every pointer to prevent memory leaks.
- Step 3 (Reset Pointers): Set inventory[i] = nullptr immediately after deletion to avoid dangling references.
- Step 4 (Reset Count): Set count = 0 to reflect an empty inventory state.